Consolidated Design Diagrams

TechniView design documentation

Techniview User Stories

Team Members: Ryan Sippy, Jason Bellerjeau, Derek Corniello

Stakeholders:

Primary Stakeholders:

- Students
- Professors

Secondary Stakeholders:

- Problem Management Team
- IT Support Team

Hidden Stakeholders:

- Students with Disabilities
- Government Education Departments and Regulators
- University Administrators

User Stories:

US-01(Primary):

As a student, I want to be able to access and run practice coding problems in a web-based environment, so that I can improve my coding skills without needing to set up a local development environment.

US-02(Primary):

As a student, I want to track my progress and performance on coding problems, so that I can identify areas where I need to improve and monitor my growth over time.

US-03(Primary):

As a professor, I want to be able to assign and manage coding problems for my students, so that I can provide them with relevant practice material and assess their understanding of the course content.

US-04(Secondary):

As a member of the problem management team, I want to be able to easily create and curate coding problems, so that I can ensure the quality and relevance of the problems available on the platform.

US-05(Hidden):

As a University Administrator, I want to ensure that the platform complies with data privacy and security regulations, so that students' personal information and coding submissions are protected.

Use Cases:

UC-01(Expands on US-01):

Title: Accessing and Running Practice Coding Problems

Primary Actor: Student

Precondition: Student has an account on the platform and is logged in.

Main Flow:

1. Student navigates to the coding problems section of the platform and selects a problem to work on.
2. The platform displays the problem statement, input/output specifications, and any relevant constraints.
3. Student writes their solution in the provided code editor and submits it for evaluation.
4. The platform compiles and runs the student's code against a set of test cases.
5. The platform provides feedback on the student's submission, including whether it passed or failed the test cases and any relevant error messages.

Alternative Flow: Student chooses to view hints or solutions for the problem, which may provide guidance on how to approach the problem or reveal the correct solution.

Exception Flow: If the student's code fails to compile or run, the platform will display an error message and allow the student to correct their code and resubmit it.

Postcondition: Student receives feedback on their submission and can choose to attempt the problem again or move on to a different problem.

AC-01.1:

Given a student is logged in and has selected a coding problem, when they submit their code, then the platform should compile and run the code against the provided test cases and return feedback on the submission.

TechniView D0: High-level system design

Title, goal statement, and conventions

Title: TechniView Design Diagram

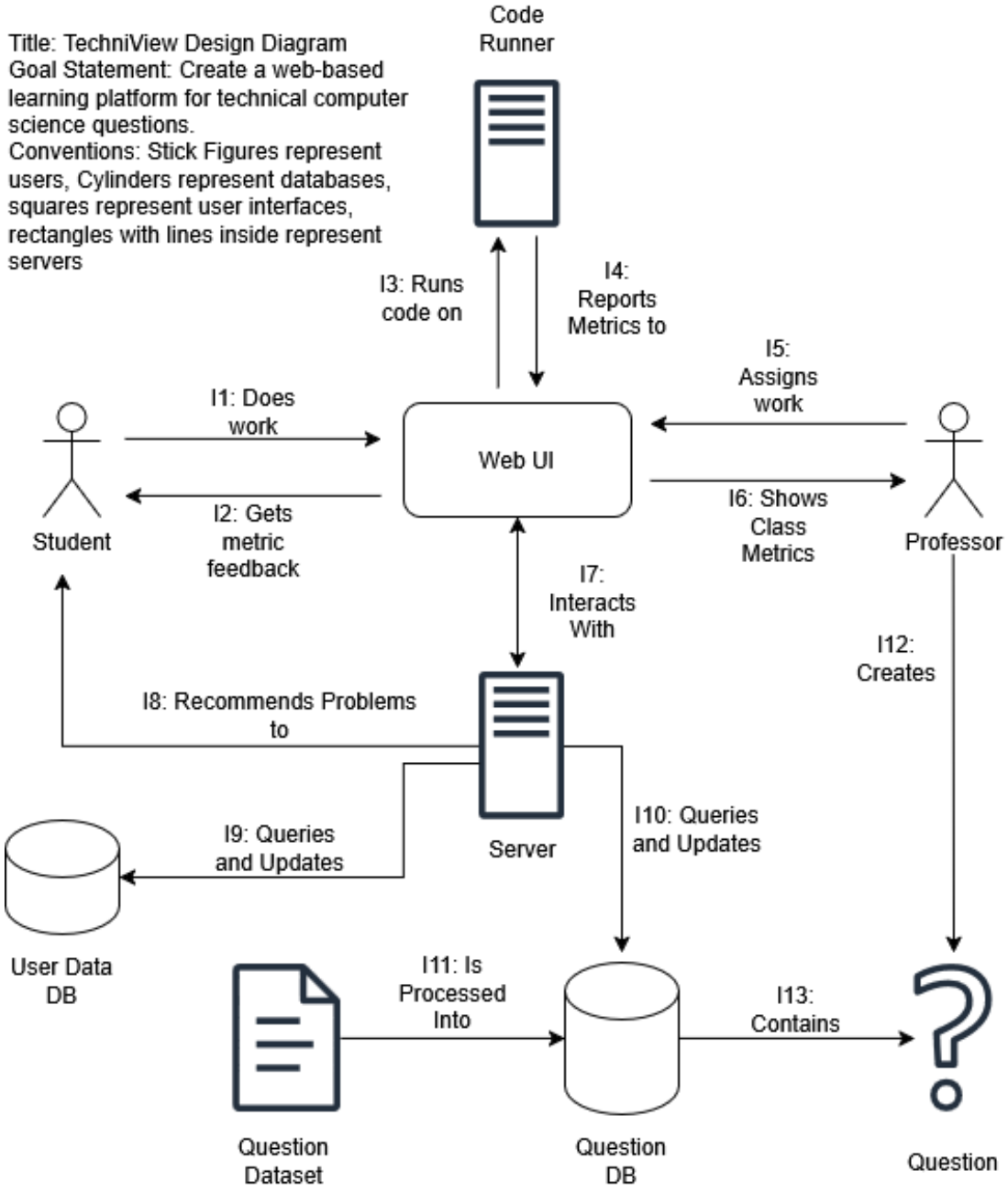
Goal Statement: Create a web-based learning platform for technical computer science questions.

Conventions: Stick Figures represent users, Cylinders represent databases, squares represent user interfaces, rectangles with lines inside represent servers

Implementation note: The User Data DB and Question DB cylinders show logical data groups. The current deployment stores both groups in the platform MySQL database. Judge0 runs separately and has its own PostgreSQL and Redis services.

Block diagram

Figure D0.1. TechniView Design Diagram. Goal: create a web-based learning platform for technical computer science questions. Stick figures represent users, cylinders represent logical data groups, squares represent interfaces, and lined rectangles represent servers.



Component responsibility table

Component	Responsibility	Interfaces In	Interfaces Out	Owner
User Data DB	Maintains all user data	Backend Server	Backend Server	Jason Bellerjeau
Question DB	Stores all question data	Backend Server, Question Dataset	Backend Server	Jason Bellerjeau
Backend Server	Handles queries to databases	Web UI, User Data DB, Question DB	Web UI, User Data DB, Question DB	Derek Corniello
Web UI	Handles interaction with user	Backend Server, Code Runner	Backend Server, Code Runner	Ryan Sippy
Code Runner	Runs each problems code	Web UI	Web UI	Derek Corniello

Interface specification table

Interface	Input	Output	Data Format	Protocol	On Error	Handling Component	Ex. Payload
I1	Student interaction	Submission to Web UI	JSON	HTTPS	Display UI Error Message	Web UI	{“student_id”: 1, “code”:”print(‘hello’)”}
I2	Metrics from Web UI	Visual Feedback	JSON	HTTPS	Display system error	Student Client	{“status”: “pass”}
I3	Execution request	Code sent to runner	JSON	REST	Return Error	Code Runner	{“source_code”: “...”}
I4	Results from runner	Metrics sent to Web UI	JSON	REST	Log failure	Web UI	{“tests_passed”: 5}
I5	Assignments from Professor	Assignments sent to Web UI	JSON	HTTPS	Display error in UI	Web UI	{“assignment_id” : 1, “problem_id” : Q101}
I6	Aggregated Data	Rendered Dashboard	JSON	HTTPS	Display Data Unavailable error	Professor Client	{"avg_score": 88.5, "completion": 0.95}
I7	API requests	API response	JSON	HTTPS	Return HTTP codes	Server	{"action": "fetch_assignments"}
I8	Performance Data	Suggestions	JSON	HTTPS	Return recommendation error	Server	{"recommended_ids": ["Q10", "Q12"]}
I9	User Data request	User Data results	SQL	TCP	Log DB error	User Data DB	“SELECT * FROM users WHERE id='101';”
I10	Read/Write requests	Question data	SQL	TCP	Log DB Error	Question DB	“SELECT prompt FROM questions WHERE id='Q1';”
I11	Raw question files	Database records	CSV, SQL	Script	Halt Pipeline	Question DB	[{"id":"Q1", "text":"Sort array"}]
I12	Text question input	Database record	Text	HTTPS	Display Error on client	Web UI/Server	{"title": "Binary Tree", "difficulty": "Hard"}
I13	Database record	Question	SQL	TCP	Throw error	Server	Question(id="Q1", type="Algorithm")

Data flow diagrams

Figure D0.2. TechniView Design Diagram. Goal: create a web-based learning platform for technical computer science questions. Boxes name services or actors, arrows show data flow, and arrow labels name the exchanged data.

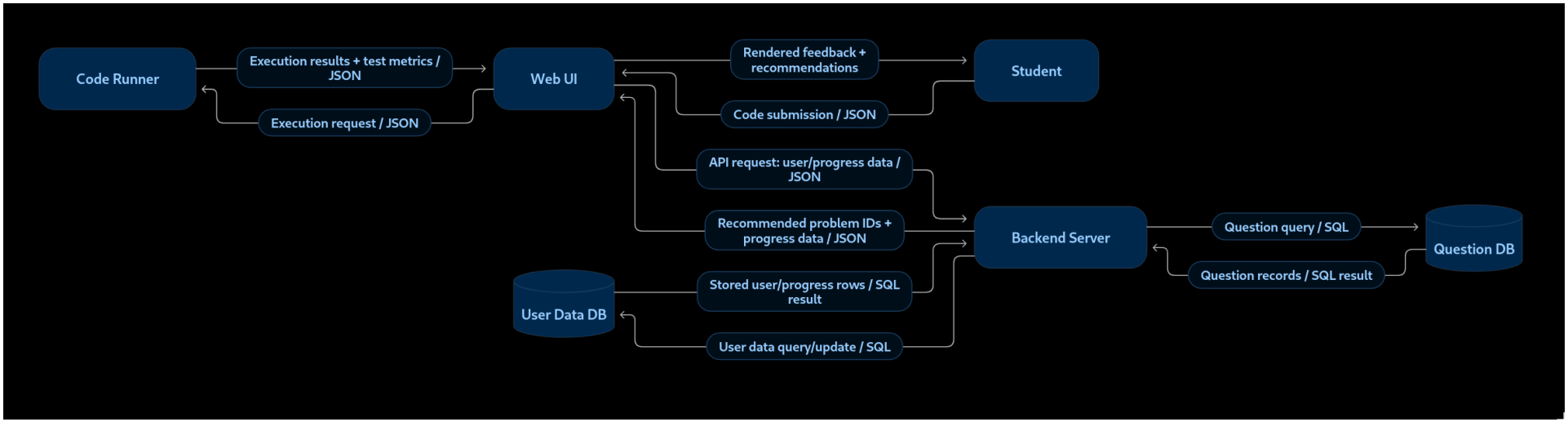
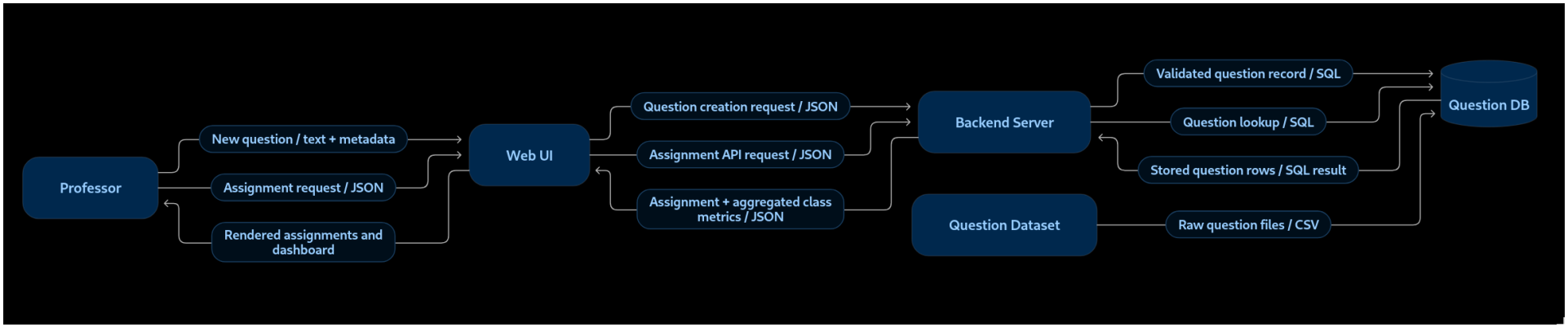


Figure D0.3. TechniView Design Diagram. Goal: create a web-based learning platform for technical computer science questions. Boxes name services or stores, arrows show data flow, and arrow labels name the exchanged data.



Architecture pattern and justification

The system is a combination of client-server, microservices, and pipeline architectural patterns. The primary interaction for the users is governed by a client-service pattern, with the Web UI being the client communicating with backend servers. The backend is governed by microservices with the general backend server being separated from the code runner. We chose to separate them because it was the right “fit to the problem”. Isolating the code runner ensures that malicious code does not impact the main server or web UI. Finally, we chose to use a pipeline to govern the data ingestion process to process the question dataset into the question DB. We rejected an embedded sensor-actuator pattern because that is primarily for hardware systems, whereas our systems is entirely web-based, so it is not the right fit.

Decision log

Decision	Alternatives Considered	Reasoning
Use a web-based client-server architecture	Desktop application with monolithic local application	A web application allows students and professors to access TechniView without installing software. The Web UI can handle UI/UX while the Backend Server has logic and database access.
Separate the Code Runner from the Backend Server	Execute submitted code directly on the Backend Server and execute code in the browser	The Code Runner isolates student-submitted code from the primary application server. This reduces the risk that incorrect or malicious code can affect the Server or UI and allows code execution to be managed independently in a containerized environment.
Use separate User Data and Question databases	Store all application data in one database and store question data in static files	User progress and question content serve different purposes and have different usage patterns. Separating them keeps their responsibilities clear and allows each data source to be queried and maintained independently.
Use JSON/REST for application communication	Direct database access from the Web UI, and/or custom binary protocol with form-encoded messages	JSON over HTTP/REST is simple to implement in a web application, easy to inspect during development, and provides a clear interface between the UI, Backend Server, and Runner.
Use a question-ingestion pipeline for dataset questions	Enter every question manually, allow the application to read raw dataset files directly	A pipeline takes raw questions into a consistent database representation before they are used by the app. This keeps data processing separate from application requests and prevents the Backend Server from repeatedly parsing inputs.

TechniView D1: Detailed design

Header, scope, and conventions

Project title: TechniView Design Diagram
Goal statement: Create a web-based learning platform for technical computer science questions.

This document details the Database component and its submission grading and assignment progress computations. It defers the Backend Server's remaining routes, the Code Runner internals, and the Web UI to D2. The design is based on repository commit `2257305`. The User Data DB and Question DB are table groups in one MySQL 8.0 database; the Backend Server calls the repository's internal Judge0 fork.

This is the D1 design snapshot. The implementation has since added course enrollment, problem authoring, question sets, assignment recipients, curriculum, staff submission review, and health routes. D2 records the current implementation and its gaps.

The D0 figure uses stick figures for users, cylinders for databases, squares for user interfaces, and rectangles with inner lines for servers. In the D1 data model, each box is a table containing fields that affect behavior. Dashed boxes are Question DB tables; solid boxes are User Data DB tables. Lines are foreign-key relationships, and their labels name the relationship. `PK` means primary key, `FK` means foreign key, `UQ` marks a field in a unique constraint, and `IX` marks an indexed field. A `UQ` or `IX` mark may be part of a composite key; the index table gives the exact columns. Crow's-foot marks show cardinality at both ends: a bar means one, a circle means zero, and a three-pronged foot means many.

Data model

Figure 1 shows the 10 tables selected for grading and assignment progress from the larger ORM model. It supports US-01 (submission and feedback), US-02 (student progress), and US-03 (assignments and assessment). The compact drawing leaves out `AssignmentItem.item_order` and `StudentAssignmentProgress.retry_count`. `item_order` controls question order. The current grader gives every test case equal weight. Practice progress is stored in `StudentPracticeProgress`, which is outside this assignment-focused figure.

The relational model fits because grading relies on foreign keys, unique constraints, transactions, and row locks. The composite foreign key from `submissions` to `assignment_items` requires an assigned submission's `problem_id` to match the item's `problem_id`. Foreign keys also protect submission and test-case references; the service selects cases from the submitted problem. Row locks protect callback and progress updates. A document store would move more of these checks into application code.

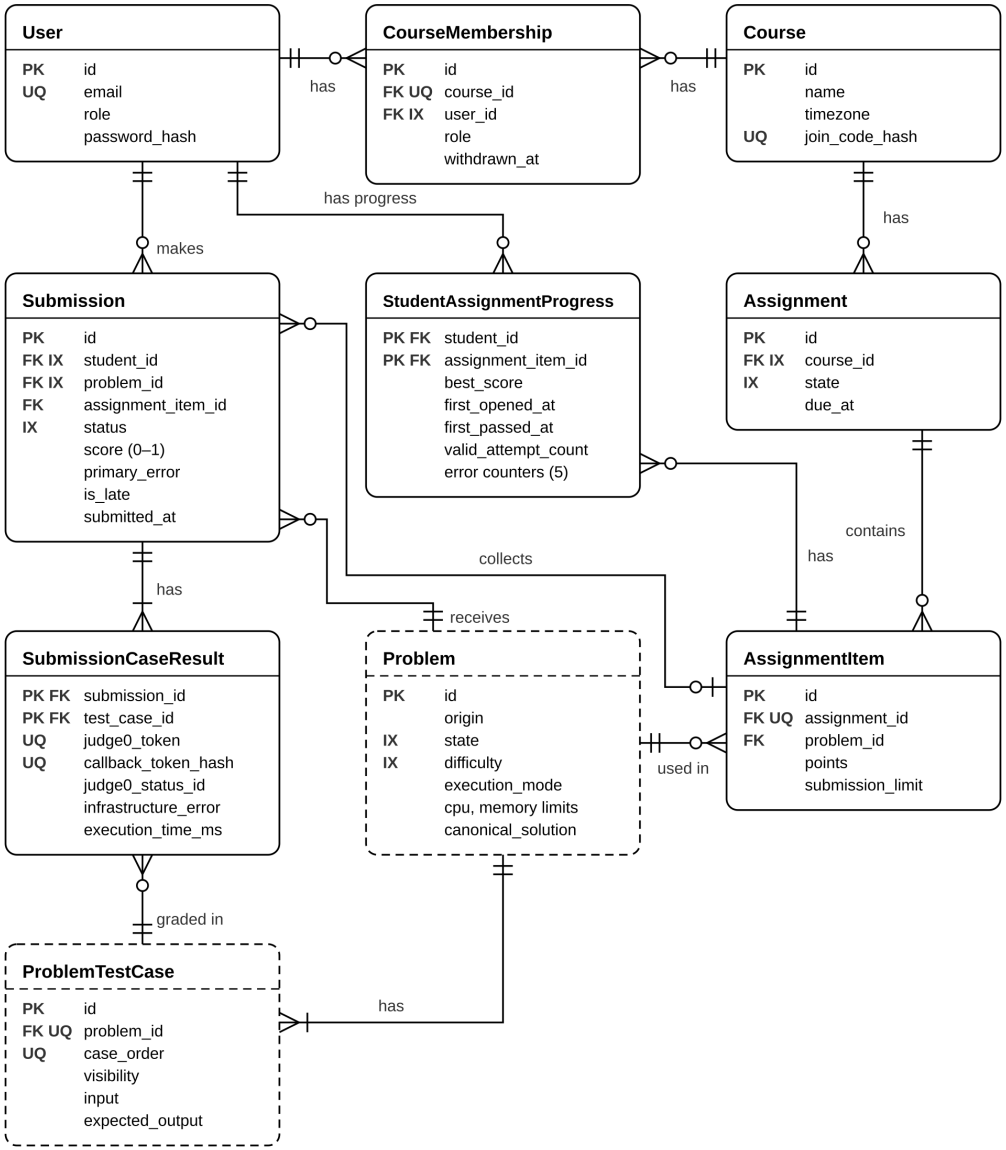
Figure 1. D1 data model

Diagram title: TechniView D1 data model. **Project title:** TechniView Design Diagram. **Goal:** create a web-based learning platform for technical computer science questions. **Conventions:** each box is a table; solid boxes are User Data DB tables and dashed boxes are Question DB tables; labeled lines are foreign-key relationships; PK, FK, UQ, and IX mark keys and indexes; crow's-foot marks show cardinality.

The editable source is the SVG.

Each submission fans out to one Judge0 result per test case

Dashed boxes: D0 Question DB tables. Solid boxes: D0 User Data DB tables. One MySQL 8.0 database.
PK primary key; FK foreign key; UQ unique constraint; IX indexed. Composite columns are listed below.




```
erDiagram
    USER ||--o{ COURSE_MEMBERSHIP : has
    COURSE ||--o{ COURSE_MEMBERSHIP : has
```

```
USER ||--o{ SUBMISSION : makes
PROBLEM ||--o{ SUBMISSION : receives
ASSIGNMENT ||--o{ ASSIGNMENT_ITEM : contains
PROBLEM ||--o{ ASSIGNMENT_ITEM : used_in
ASSIGNMENT_ITEM o|--o{ SUBMISSION : answers
SUBMISSION ||--o{ SUBMISSION_CASE_RESULT : has
PROBLEM_TEST_CASE ||--o{ SUBMISSION_CASE_RESULT : graded_in
USER ||--o{ STUDENT_ASSIGNMENT_PROGRESS : has
ASSIGNMENT_ITEM ||--o{ STUDENT_ASSIGNMENT_PROGRESS : collects

USER {
    int id PK
    string email UQ
    string role
    string password_hash
}
COURSE {
    int id PK
    string name
    string timezone
    string join_code_hash UQ
}
COURSE_MEMBERSHIP {
    int id PK
    int course_id FK
    int user_id FK
    string role
    datetime withdrawn_at
}
PROBLEM {
    int id PK
    string origin
    string state IX
    string difficulty IX
    string execution_mode
    decimal cpu_time_limit_seconds
    int memory_limit_kb
    string canonical_solution
}
PROBLEM_TEST_CASE {
    int id PK
    int problem_id FK
    int case_order UQ
    string visibility
    string input
    string expected_output
}
ASSIGNMENT {
    int id PK
    int course_id FK
    string state IX
    datetime due_at
}
ASSIGNMENT_ITEM {
    int id PK
    int assignment_id FK
    int problem_id FK
    int item_order
    decimal points
    int submission_limit
}
SUBMISSION {
    int id PK
    int student_id FK
    int problem_id FK
    int assignment_item_id FK
    string status IX
    decimal score
    boolean is_late
    datetime submitted_at
}
SUBMISSION_CASE_RESULT {
    int submission_id PK, FK
    int test_case_id PK, FK
    string judge0_token UQ
    string callback_token_hash UQ
```

```
        int judge0_status_id
        boolean infrastructure_error
        decimal execution_time_ms
    }
    STUDENT_ASSIGNMENT_PROGRESS {
        int student_id PK, FK
        int assignment_item_id PK, FK
        decimal best_score
        datetime first_opened_at
        datetime first_passed_at
        int valid_attempt_count
        int retry_count
        int wrong_answer_count
        int compile_error_count
        int runtime_error_count
        int timeout_count
        int resource_limit_count
    }
```

Structural decision	Choice	Reason and requirement
Course roles	`CourseMembership` entity, not a role attribute on `User`	A person can have different roles in different courses. `withdrawn_at` keeps membership history. US-03.
Problems in an assignment	`AssignmentItem` entity, not a bare link	It holds points, order, and submission limit. US-03.
Per-test results	`SubmissionCaseResult` entity, not a JSON column	Judge0 reports cases separately; each row stores its own token and verdict. US-01, AC-01.1.
Progress	`StudentAssignmentProgress` row, updated on finalization	Dashboard reads one row per student and assignment item. US-02, US-03.
Submission to assignment item	A submission references zero or one assignment item; an item can have many submissions	A submission answers at most one assigned problem. Practice submissions leave the reference null. US-01, US-03.
Difficulty and visibility	Enum attributes, not entities	These are fixed values with no independent data or relationships. US-01.
Storage model	Relational MySQL 8.0	Foreign keys, unique constraints, transactions, and row locks enforce grading integrity. US-01, US-02.

Indexed or unique columns	Why
`users(email)` unique	Login lookup and duplicate account prevention.
`course_memberships(course_id, user_id)` unique; `course_id` and `user_id` indexed	Enforce one membership per user and course; support course rosters and reverse lookup by user.
`problem_test_cases(problem_id, case_order)` unique	Load a problem's cases in stable order and prevent duplicate case positions.
`assignments(course_id)` and `assignments(state)`	Filter assignments by course and state.
`problems(state)` and `problems(difficulty)`	Filter the problem bank by publication state and difficulty.
`submissions(student_id, problem_id)`	Filter a student's submissions by problem.
`submissions(status)`	Filter submissions by queued, running, or terminal state.
`assignment_items(assignment_id, problem_id)` unique	Prevent duplicate problems in one assignment.
`assignment_items(assignment_id, item_order)` unique	Prevent duplicate positions in an assignment.
`assignment_items(id, problem_id)` unique	Support the composite foreign key from a submission to its assignment item.
`submissions(id, problem_id)` unique	Provide a unique submission/problem pair for related records.
`student_practice_progress(student_id, problem_id)` primary key	Find or lock one student's practice progress row.
`submission_case_results(judge0_token)` unique	Match a Judge0 report to its case row.
`submission_case_results(callback_token_hash)` unique	Find and authenticate callbacks without storing the callback secret.

Indexed or unique columns	Why
`student_assignment_progress(student_id, assignment_item_id)` primary key	Find or lock one student's progress row.
`user_sessions(expires_at)`	Find expired sessions for cleanup.

Core algorithms

Planning size: 500 problems with 8,371 test cases, about 16.7 cases per problem on average, median 10, maximum 122, 50 students and 30 assignment items per course, and a peak of about 20 submissions per minute. These are estimates, not measured load. US-01 and AC-01.1 drive grading. US-02 and US-03 drive progress reporting.

Algorithm 1: Submission grading

Purpose. Run a student's Python code against every test case in Judge0 and turn the reports into one status and score. A wrong verdict produces a wrong grade (US-01, AC-01.1).

Inputs and outputs. `problem_id: int` must be greater than zero. `assignment_item_id: int | None` is optional and, when present, must be greater than zero. `source_code: str` must contain 1 to 65,535 UTF-8 bytes. `wait: bool` is an optional query parameter that defaults to false. The server loads the problem's test cases, CPU limit (`Decimal` seconds, default 2.0), and memory limit (`int` KB, default 128,000). It creates a `Submission` with an enum status, nullable decimal `score` from 0 to 1, nullable `primary_error`, and one `SubmissionCaseResult` per test case.

Steps. The server checks problem visibility. For an assignment item, it verifies that the item is published, available, and assigned to the caller. It enforces the attempt limit, creates or locks the progress row, records whether the submission is late, and inserts the queued submission. It creates one result row per test case with a random callback token stored as a SHA-256 hash. It then sends one HTTP request per case to Judge0, in sequence. Each job runs asynchronously on Judge0 workers. Call-based problems use a wrapper that calls the student's function and prints JSON. A callback locks and updates its case row. When all cases finish, the server sets status and score. Each case has equal weight. The score is `passed_case_count / total_case_count`. Failure priority is compilation error, timeout, resource limit, runtime error, then wrong answer.

Complexity and scale. With `T` test cases, the server makes `T` sequential Judge0 submission calls and creates or updates `O(T)` result rows. Total work is `O(T)` plus code execution. At the planning peak, the catalog average means about 335 Judge0 jobs per minute; a 122-case problem can produce 2,440 jobs per minute if every submission uses that problem. Submission latency includes the sum of the sequential HTTP request times; execution and callback time depend on Judge0's queue. At 100 times the estimated submission rate, the average becomes about 33,500 jobs per minute. That growth matters, but capacity has not been measured. Measure request latency and queue depth before choosing concurrent submissions, batch requests, or more workers.

Why this approach. Judge0 handles the security-sensitive sandbox, which the team has no experience building. A separate job per case gives each case its own limits and verdict. Callbacks deliver results after the browser closes. `GET /api/submissions/{id}` is the polling fallback.

Edge cases. Empty source is rejected with 422. A problem with no test cases returns 409. Duplicate callbacks and callbacks received after finalization cannot change the final grade. Failure ties use the fixed priority above. Judge0 failures return 429, 502, or 504; infrastructure failures do not count against the student's attempt limit.

Algorithm 2: Progress aggregation

Purpose. For an assignment submission, update the student's `StudentAssignmentProgress` row, then summarize assignment progress for professors (D0 interface I6, US-02, and US-03). Practice submissions update `StudentPracticeProgress` through the same finalization path. Professor course analytics read assignment progress only.

Inputs and outputs. The update input is a finalized `Submission` and its locked `StudentAssignmentProgress` row, or `StudentPracticeProgress` row when `assignment_item_id` is null. The row stores `best_score: Decimal`, `first_opened_at: datetime | None`, `first_passed_at: datetime | None`, `valid_attempt_count: int`, `retry_count: int`, and five integer error counters. The analytics read accepts `course_id: int`, optional `assignment_id: int`, optional `difficulty: easy | medium | hard`, optional `tag: str`, `limit: int` from 1 to 100 (default 50), and `offset: int` >= 0 (default 0).

The response's `summary` and each item's `metrics` have integer `assigned_count`, `started_count`, `completed_count`, `total_valid_attempts`, and `total_retries`; `completion_rate: float | None` as a ratio from 0 to 1; `avg_completion_seconds` and `median_completion_seconds` as seconds or null; `avg_retries_per_student: float | None`; and `errors` with integer `wrong_answer`, `compile_error`, `runtime_error`, `timeout`, and `resource_limit` counters. Each item is `{problem: ProblemSummary, metrics: Metrics}`. The outer response

also has `items`, `total`, `limit`, and `offset`.

Steps. Finalization skips infrastructure errors, increments the valid attempt and matching error counter, and updates `retry_count`. The code updates `best_score` only for on-time or approved-late work and sets `first_passed_at` on the first eligible passing submission. On analytics reads, the server joins assignment items to recipients and membership rows, left-joins progress, and includes non-draft assignments. It filters difficulty and tag, groups metrics by problem in Python, sorts by title and ID, then slices the result for `limit` and `offset`.

Complexity and scale. The update is `O(1)` after the matching progress row is found. A course overview reads `R = assignment items × recipients` records, then groups them and sorts the problem summaries: `O(R + P log P)`, where `P` is the number of distinct problems. At the planning size, `R` is about 1,500. At 100 times that size, it is about 150,000. The larger read may take seconds with grouping in Python; that is a projection, not a measurement. Pagination slices after loading and sorting all records. The ORM model has analytics rollup tables if profiling shows this query is too slow.

Why this approach. Write-time counters avoid recounting every submission for each dashboard read. `best_score` means a retry cannot lower a student's recorded best result.

Edge cases. With no assigned rows, counts are zero and rates and completion times are null. Repeated callbacks do not double-count a finalized submission. Equal best scores leave the existing value unchanged. Draft assignments are excluded; archived assignments are included. Withdrawn students are currently included in course analytics because the query does not filter them. Confirm that roster rule before D2. A late pass does not count toward `best_score` or completion time unless it is approved.

Build-versus-reuse decisions

Piece	Build or reuse	Library or service	License	Maturity, performance, and fit	Reason
Code sandbox	Reuse	Internal Judge0 fork, upstream 1.13.1	GPL-3.0	Established judge; worker execution enforces CPU and memory limits; requires privileged containers.	The team knows REST integration, not sandbox security. Keep it separate and review obligations before redistribution.
Database	Reuse	MySQL 8.0	GPL-2.0	Mature server with transactions, row locks, and composite keys; fits the estimated course load.	The project uses it for application data, separately from Judge0's PostgreSQL.
ORM and migrations	Reuse	SQLAlchemy 2, Alembic, PyMySQL	MIT	Established Python tools; synchronous transactions and indexed queries fit the current load.	The codebase uses typed SQLAlchemy models and Alembic revisions.
API framework	Reuse	FastAPI, Uvicorn, Pydantic	MIT, BSD-3-Clause	Established Python stack; validation and generated API docs fit the contract.	Reuse for request validation and JSON API documentation.
Password hashing and dates	Reuse	Python `hashlib.scrypt`, `datetime`, `secrets`	PSF	Standard library; avoids extra dependencies for password hashing, tokens, and UTC dates.	Do not hand-roll cryptography or date parsing.
Sessions	Build	`UserSession` model and standard library token functions	MIT for this repository	Small server-side implementation; hashed, expiring, revocable session tokens fit one API.	The code stores only a hash of the random cookie token.
Grading and progress rules	Build	TechniView code	MIT	Small data volume; rules are specific to TechniView's assignment and scoring behavior.	These rules define the product behavior in the algorithms.
Problem bank	Reuse	`backend/data/problem_catalog.json`	MIT	500 audited problems; import validates required fields and source version.	Preserve the APPS and APPS+ attribution recorded in the catalog.

API contract

The detailed endpoints use JSON under `/api` and the `techniview_session` cookie. Successful requests return HTTP 200. `POST /api/submissions` accepts an optional `wait` query parameter, default false. Query pagination uses `limit` default 50 and range 1 to 100, and `offset` default 0 and range 0 or greater.

Endpoint	Inputs	Output
`POST /api/submissions`	JSON body: `problem_id: int`, required, > 0; `assignment_item_id: int`	null, `optional, if present > 0;` source_code: str, `required, 1 to 65,535 UTF-8 bytes. Extra fields are rejected. Optional query` wait: bool`, default false.
`GET /api/submissions/{submission_id}`	Path `submission_id: int`, required. IDs for existing submissions are positive; missing, non-owned, or other values return 404. Session cookie required.	HTTP 200 and the current `Submission` response, visible only to its owner.
`GET /api/courses/{course_id}/analytics/overview`	Path `course_id: int`, required, integer. Optional query `assignment_id: int`; `difficulty: easy	medium	hard`, `origin: imported	custom`, `state: draft	published	archived`, `tag: string`, `tag_kind: general	technique	problem_type`.	`Metrics`

`Submission` is `{id: int, status: SubmissionStatus, done: bool, score: Decimal | null, is_late: bool, late_approved: bool, passed_count: int, test_count: int, public_results: PublicResult[], poll_url: str}`. `SubmissionStatus` is `queued | running | passed | failed | compile_error | runtime_error | timeout | resource_limit | infrastructure_error`. `PublicResult` is `{case_order: int, status_id: int | null, friendly_message: str}`; status IDs are Judge0 values 1 through 14. `passed_count` and `test_count` include hidden tests, but `public_results` contains public cases only.

`Metrics` is `{assigned_count: int, started_count: int, completed_count: int, completion_rate: float | null, avg_completion_seconds: float | null, median_completion_seconds: float | null, total_valid_attempts: int, total_retries: int, avg_retries_per_student: float | null, errors: ErrorCounts}`. `ErrorCounts` is `{wrong_answer: int, compile_error: int, runtime_error: int, timeout: int, resource_limit: int}`. `ProblemMetrics` is `{problem: ProblemSummary, metrics: Metrics}`. `ProblemSummary` is `{id: int, title: string, difficulty: easy|medium|hard, origin: imported|custom, state: draft|published|archived, tags: string[], typed_tags: ProblemTag[], source_category: string | null}`. `ProblemTag` is `{name: string, slug: string, kind: general|technique|problem_type}`. Completion times use seconds; rates are ratios from 0 to 1.

HTTP status	Code or body	When it happens
401	`authentication_required` or `invalid_session`	Missing or expired session cookie.
403	`staff_required`	A student or non-staff member calls analytics.
404	`course_not_found`, `problem_not_found`, `assignment_item_not_found`, or `submission_not_found`	Resource is missing, unavailable to caller, or not owned by caller.
409	`problem_has_no_tests` or `submission_limit_reached`	Problem has no cases or the assignment attempt limit is reached.
422	`validation_error`	Missing field, invalid type or range, extra field, or source exceeds the byte limit.
429	`{"detail":"runner busy, try again"}`	Judge0 rejects a request because it is busy.
502	`{"detail":"runner unavailable"}`	Judge0 is unavailable or returns an invalid response.
504	`{"detail":"runner timed out"}`	Judge0 does not respond before the request timeout.

`ApiError` and request-validation failures use `{error: {code: string, message: string, details: object}}`; validation details include field diagnostics. Runner errors use FastAPI's `{detail: string}` body. A runner failure marks the reserved submission `infrastructure_error`, which does not count toward the assignment attempt limit.

Example request:

```
POST /api/submissions
Content-Type: application/json
Cookie: techniview_session=<session>
```

{"problem_id":1,"assignment_item_id":null,"source_code":"def sum_list(values):\\n   return sum(values)"}

Example response:


```
{
  "id": 42,
  "status": "queued",
  "done": false,
  "score": null,
  "is_late": false,
  "late_approved": false,
  "passed_count": 0,
  "test_count": 2,
  "public_results": [
    {"case_order": 1, "status_id": 1, "friendly_message": "In queue. Waiting for a worker."}
  ],
  "poll_url": "/api/submissions/42"
}
```


Versioning: routes will move under `/api/v1` before external clients depend on them, while database changes use Alembic revisions. Removing or renaming a response field, changing a field's type or unit, or making an optional input required is a breaking API change.

Technology choices with justification

Database: MySQL 8.0. Team skill fit: the team used MySQL in its databases course. License: GPL-2.0 for the server distribution. Community support: MySQL has a large user and operator base. Performance: transactions, row locks, composite keys, and indexed lookups fit the expected course workload. Cost and hosting: the server is free to run in a container on one project VM. We chose it over PostgreSQL for application data because the Judge0 stack already runs its own PostgreSQL service, which keeps application data separate from Judge0 state.

Backend: Python 3.14 with FastAPI. Team skill fit: Python is the team's shared language and the language students submit. License: Python uses the PSF license; FastAPI and Pydantic use MIT; Uvicorn uses BSD-3-Clause. Community support: both have active ecosystems, and FastAPI builds API documentation from schemas. Performance: request processing is small compared with code execution in Judge0. Cost and hosting: the software is free and runs in the existing backend container. We chose this over Node.js with Express so the API and grading wrappers use one language.

Front end: React, TypeScript, and Vite. Team skill fit: team members have used React in prior projects. License: React and Vite use MIT; TypeScript uses Apache-2.0. Community support: all three have broad adoption and documentation. Performance: Vite builds static assets; expected course pages do not need server-side rendering. Cost and hosting: the assets can be served from the same VM. The current Compose setup runs Vite's development server; production should serve the built assets. We chose this over a server-rendered Python front end because the team has more React experience.

Code execution and messaging: internal Judge0 fork with HTTP callbacks. Team skill fit: the team knows how to call REST services, but has no experience building a secure sandbox. License: the fork is GPL-3.0; it is isolated as a service, and redistribution obligations must be reviewed before distribution. Community support: the fork is based on the established Judge0 1.13.1 release. Performance: worker execution applies per-case CPU and memory limits, but peak capacity needs measurement. Cost and hosting: the software is free, though workers require privileged containers and may need a separate VM as load grows. The app sends one request per case and receives callbacks; Redis and PostgreSQL are internal Judge0 dependencies, not a TechniView job queue.

Hosting: Docker Compose on one Linux VM. Team skill fit: the team uses Compose to start the same services locally and on the server. License: Docker Compose is Apache-2.0; the VM's Linux distribution has its own license. Community support: Compose is common for small multi-service deployments. Performance: one VM fits one or two courses under the planning estimates; move Judge0 workers to another VM if measurements show contention. Cost and hosting: a university or student-credit VM keeps cost low. We chose this over Kubernetes because this project does not need a cluster or its additional operating work.

TechniView D2: Detailed design

Header, conventions, and D2 overview

Project title: TechniView Design Diagram
Goal statement: Create a web-based learning platform for technical computer science questions.

Diagram conventions: Actors are stick figures. Rounded boxes are browser or application components. Rectangles are services. Cylinders are persistent stores. Arrows show request or data flow; arrows labeled with an error show failure paths. In data models, boxes name MySQL tables, lines show foreign-key relationships, PK means primary key, FK means foreign key, and UQ means a unique constraint. An item marked “no persistent data” owns no application table.

Two-minute overview. TechniView is a course-linked coding practice platform. Students browse Python problems, submit code, see test results, and review their progress. Professors manage courses, curate problems, build question sets, publish assignments, review submissions, and view analytics. The FastAPI backend stores application data in MySQL. It sends isolated code jobs to the internal Judge0 fork. Judge0 uses its own PostgreSQL and Redis. D1 designed the grading and progress data model, the grading and aggregation algorithms, and selected API contracts. D2 accounts for the rest of the current backend, frontend, operations, and integration boundaries, then records risks and open questions.

The repository implements many of these flows, but implementation status varies by component. The React app has login, student, and teacher routes. Problem authoring, assignments, and class workflows have backend APIs, but the visible pages are not a complete workflow for every API. Notification delivery does not exist. The backend has no application-owned audit log or structured logging policy. Health endpoints probe dependencies, but they are not an alerting system.

Known gaps and open questions

- The user stories name privacy and regulatory compliance, but no jurisdiction, retention schedule, consent rule, or acceptance threshold is specified.
- Production cookie security is configurable and defaults to false. Production CORS origins, HTTPS termination, secret management, backups, and restore targets are not defined in this repository.
- No notification service or notification preferences exist. Assignment publication and late approval do not send messages.
- No session cleanup job, rate limiting, account recovery flow, or MFA exists.
- Course membership removal is a soft withdrawal. Assignment recipients are snapshotted at assignment creation, and analytics can continue to include a withdrawn recipient.
- Large analytics queries load records and slice after grouping. Pagination is not pushed into the database.
- The public frontend does not expose all backend workflows. Some teacher and student routes are present, but product completeness and accessibility have not been audited.
- External peer-team comments were not supplied with the repository. The companion review notes identify code and document audit findings separately from feedback that must be collected in class.

Finished components, the D2 diagrams

Current component map

Component	Current data model	Current behavior	Interface
Browser clients	No persistent data. React state only.	Auth provider checks the session; role routes direct students and professors to their pages. Pages call the API with cookies.	React routes listed below; HTTPS/JSON in deployment.
Authentication	User, UserSession.	Scrypt password verification; random session token returned in an HTTP-only cookie; only its SHA-256 hash is stored. Sessions expire after configured days and logout revokes them.	POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me.
Course and membership	Course, CourseMembership.	Join links enroll or reactivate students. Staff can view rosters, change roles, and withdraw members. The last active instructor cannot be demoted or removed.	GET /api/courses, GET /api/courses/{course_id}, GET /api/courses/{course_id}/members, PATCH and DELETE member routes, GET /api/join/{token}, POST signup/login routes.
Problem catalog and curriculum	Problem, ProblemTestCase, ProblemTag, problem_tag_assignments, CurriculumNode.	Published problems are browsable under visibility rules. Public examples are returned; hidden cases and canonical solutions are withheld. Curriculum is derived from problem tags and progress.	GET /api/problems, GET /api/problems/{problem_id}, POST /api/problems/{problem_id}/start, GET /api/curriculum, GET /api/curriculum/available.
Problem authoring and question sets	Problem, ProblemTestCase, ProblemTag, QuestionSet, QuestionSetItem.	Professors create and edit owned drafts, copy visible problems, validate the canonical solution against every case, then publish or archive. Question-set items are replaced as an ordered list.	POST/PATCH /api/problems, POST copy/validate/publish/archive routes, GET/POST/PATCH /api/question-sets, PUT items, POST publish/archive.
Assignment management	Assignment, AssignmentItem, AssignmentRecipient; D1 data model.	A published owned question set is copied into a draft assignment. Active student memberships become recipients. Instructors publish non-empty drafts. Students see published, available assignments addressed to them.	POST /api/courses/{course_id}/assignments, POST publish, GET list and detail routes.
Submission grading and progress	Submission, SubmissionCaseResult, StudentAssignmentProgress, StudentPracticeProgress; detailed in D1.	The API reserves a submission, creates one Judge0 job per case, consumes callback or polling results, computes score and terminal status, then updates progress.	POST/GET /api/submissions, internal Judge0 callback route, staff submission routes under /api/courses/{course_id}/submissions.
Analytics	Student and assignment progress tables; detailed in D1.	Student and staff reads aggregate progress, errors, completion, and retries. Staff endpoints require instructor or TA membership.	GET /api/me/analytics, course overview, problem, and student-problem analytics routes.
API runtime and configuration	No persistent data. Environment variables configure MySQL, Judge0, cookie policy, CORS, and Judge0 callback URL.	FastAPI mounts routers under /api, applies CORS, dependency checks, and shared validation/API-error handlers.	JSON over HTTP; OpenAPI at /api/docs.
Platform persistence	MySQL tables described above. The logical User Data DB and Question DB are not separate MySQL services.	SQLAlchemy sessions and Alembic migrations read and write application data. Foreign keys, unique constraints, and transactions enforce relationships.	Internal SQL connections only.
Code runner	No TechniView application tables. Judge0 owns PostgreSQL and Redis state.	The internal Judge0 fork runs Python submissions in workers with per-job CPU and memory limits. Callback delivery returns case outcomes.	Internal Judge0 REST submission/status endpoints and TechniView callback endpoint.
Health and operational checks	No persistent application data.	Health handlers probe MySQL, Judge0, Judge0 PostgreSQL, and Redis. Deep Judge0 check submits a small Python program. No alert routing or structured application log store is present.	GET /api/health, /api/health/{mysql,postgres,redis,judge0}.

Component	Current data model	Current behavior	Interface
Logging	No persistent data. Uvicorn process output only; no application audit log table or configured log sink.	Uvicorn can write access/runtime output to process streams. Business actions and security events are not recorded in an application-owned log.	Process stdout/stderr for the container runtime; no application logging API.
Notifications	No persistent data. Not implemented.	No email, push, or in-app delivery. Current behavior is to expose state through API reads and UI pages.	No notification endpoint or message bus.

Browser behavior and interfaces

The browser sends JSON requests with credentials included. Authentication uses the HTTP-only techniview_session cookie, not a bearer token. The frontend API wrapper throws a generic status error for non-success responses. Login displays a generic credential error. Protected routes redirect unauthenticated users to login and restrict teacher routes to professors.

The frontend currently exposes /login, /student/statistics, /student/problems, /student/problems/{problemId}, /teacher/class, /teacher/statistics, and /teacher/problems. The API surface is broader than those routes. The browser has no durable local data model. Browser refreshes recover the user via GET /api/auth/me.

Persistence model added after D1

The following tables complete the parts of the current model not shown in D1. All are in the same platform MySQL schema.

Table	Key fields and relationships	Purpose
User	id PK, name, email UQ, password_hash, role, disabled_at	Account identity and global role.
UserSession	id PK, token_hash UQ, user_id FK, created_at, last_used_at, expires_at, revoked_at	Hashed, expiring login sessions.
Course	id PK, name, description, timezone, join_code_hash UQ, archived_at	Course identity and hashed invite link.
CourseMembership	id PK, course_id FK, user_id FK, role, joined_at, withdrawn_at; course_id + user_id UQ	Per-course membership and withdrawal history.
Problem	id PK, owner_id FK, copied_from_id FK, origin, state, title, prompt, difficulty, execution_mode, function_name, starter_code, canonical_solution, resource limits, source attribution	Built-in and professor-authored problems.
ProblemTestCase	id PK, problem_id FK, case_order, visibility, input, expected_output; problem_id + case_order UQ	Ordered public and hidden test cases.
ProblemTag and problem_tag_assignments	ProblemTag id PK, name UQ, slug UQ, kind; assignment join has problem_id + tag_id PK	Search and curriculum classification.
CurriculumNode	problem_id PK/FK, priority UQ	Explicit problem order for the baseline curriculum.
QuestionSet	id PK, owner_id FK, title, description, state, created_at	Professor-owned ordered source for assignments.
QuestionSetItem	id PK, question_set_id FK, problem_id FK, item_order; set/order and set/problem UQ	Ordered question-set contents.
Assignment	id PK, course_id FK, assigned_by_membership_id + course_id composite FK, source_question_set_id FK, title, state, availability, due date	Course assignment lifecycle.
AssignmentItem	id PK, assignment_id FK, problem_id FK, item_order, points, submission_limit	Ordered assignment problems and grading limits.
AssignmentRecipient	assignment_id + membership_id PK, course_id; composite FKs to assignment and membership	Snapshot of students assigned at creation time.

State changes. Problem: draft -> published -> archived; published problems cannot be edited, and copying creates a new draft. Question set: draft -> published -> archived; published content is immutable. Assignment: draft -> published; there is no archive route in the current API. Submission: queued -> running -> passed/failed/compile_error/runtime_error/timeout/resource_limit/infrastructure_error. A terminal submission does not accept later callback changes. Course membership moves between active and withdrawn; rejoining reactivates the same membership without changing its role.

Backend interfaces

All routes below are mounted under /api. Request and response bodies are JSON unless the route sets or clears the session cookie or returns 204.

Area	Routes	Inputs and outputs
Authentication	POST /auth/login, POST /auth/logout, GET /auth/me	Login takes email/password, sets the HTTP-only cookie, and returns User. Logout revokes the matching session if present, clears the cookie, and returns 204. Me returns the authenticated user.
Course join	GET /join/{token}, POST /join/{token}/signup, POST /join/{token}/login	Preview returns course summary. Signup takes name, email, password, confirmation. Login takes email/password. Successful signup/login enrolls and sets the session cookie.
Courses	GET /courses, GET /courses/{id}, GET /courses/{id}/members, PATCH /courses/{id}/members/{membership_id}, DELETE same member path	List/member routes accept bounded limit/offset. Role update accepts a membership role. Removal is soft withdrawal and returns 204.
Problem reads	GET /problems, GET /problems/{id}, POST /problems/{id}/start	List filters difficulty, tag, and pagination. Detail returns problem statement and public test cases only. Start records first-opened progress and returns 204.
Problem authoring	POST/PATCH /problems, POST /problems/{id}/copy, /validate, /publish, /archive	Draft payload includes title, prompt, difficulty, execution mode, function name, starter, optional canonical solution, limits, tags, and ordered cases. Publish validates the canonical solution through Judge0.
Question sets	GET/POST /question-sets, GET/PATCH /question-sets/{id}, PUT /question-sets/{id}/items, POST /publish, POST /archive	Professor-owned set; items request replaces ordered problem_ids. Duplicate or unavailable problems return a client error.
Assignments	POST /courses/{id}/assignments, POST /courses/{id}/assignments/{assignment_id}/publish, GET list/detail	Creation requires a published owned question set and title; optional available_at and due_at must be ordered. Publishing requires at least one item.
Submissions	POST /submissions, GET /submissions/{id}	POST takes problem_id, optional assignment_item_id, source_code, optional wait. Source is 1 to 65,535 UTF-8 bytes. GET is owner-only and polls Judge0 when results are not terminal.
Staff submissions	GET /courses/{id}/submissions, GET /courses/{id}/submissions/{submission_id}, POST /courses/{id}/submissions/{submission_id}/approve-late	Requires staff membership. Late approval records approving membership and timestamp.
Analytics	GET /me/analytics, GET /courses/{id}/analytics/overview, GET /courses/{id}/analytics/problems/{problem_id}, GET /courses/{id}/analytics/students/{student_id}/problems/{problem_id}	Optional difficulty/tag and assignment filters. Overview accepts limit 1..100 and offset >= 0. Staff reads require staff membership.
Curriculum	GET /curriculum, GET /curriculum/available	Optional kind selects technique or problem-type view. Returns pool metadata or available problem summaries for authenticated students.
Internal runner	PUT /internal/judge0/callbacks/{callback_token}	Judge0 posts a case report. Token is matched by hash; unknown or mismatched callback returns an error. This route is omitted from public OpenAPI.
Health	GET /health, /health/mysql, /health/postgres, /health/redis, /health/judge0	Returns dependency status, latency, and limited version/diagnostic fields. Deep Judge0 health submits print(42).

D0 interface disposition

The old I1-I13 labels are conceptual D0 connections. The current interface below is narrower and avoids sending internal IDs, hidden test data, or SQL details to browser clients.

D0 interface	Current boundary and contract	Revision
I1 Student interaction	Browser to authenticated API. POST /submissions accepts problem_id, optional assignment_item_id, and source_code.	The API derives student_id from the session rather than trusting a client-supplied identity.
I2 Metrics to student	API returns submission status, score, public case feedback, and GET /me/analytics metrics.	Keep hidden expected outputs and canonical solutions out of student responses.
I3 Execution request	Backend to internal Judge0 REST call, one job per test case with execution limits and callback URL.	Browser no longer calls the runner directly. Runner details stay behind the backend.
I4 Execution result	Judge0 callback posts a case report to the internal callback route; GET submission polls status as fallback.	Replace direct runner-to-browser metrics with authenticated backend state.
I5 Professor assignment	Professor creates an assignment from a published owned question_set_id with title and optional dates.	Problem list/order is copied from the question set; the caller does not supply arbitrary course membership or recipient IDs.
I6 Professor metrics	Staff calls course analytics overview/problem/student-problem routes.	Role and course membership are checked server-side; return aggregate fields rather than database rows.
I7 General API request	Browser uses JSON endpoints under /api with the HTTP-only session cookie.	Replace generic action payload with typed route-specific schemas and stable ApiError codes.
I8 Recommendations	GET /curriculum and /curriculum/available return pool metadata and available summaries using tags and progress.	Keep recommendation rules in the curriculum service; do not expose database queries. No personalized recommendation endpoint exists.
I9 User data access	Backend SQLAlchemy repository/session accesses application tables in MySQL.	SQL is internal and parameterized; browser never receives SQL or database credentials.
I10 Question data access	Backend SQLAlchemy repository/session reads and writes problem/question-set tables in the same platform MySQL schema.	Treat Question DB as a logical data group, not a separate deployed application database.
I11 Dataset import	Operator runs the catalog import command against a validated JSON catalog.	This is an offline operator workflow, not a client upload endpoint.
I12 Question authoring	Professor sends typed ProblemInput or QuestionSet input to authoring routes.	Canonical solution and hidden cases are professor-only; published content is immutable.
I13 Internal question object	Backend services pass SQLAlchemy Problem/ProblemTestCase records and response projections internally.	ORM objects are not exposed as a public wire format; response schemas select allowed fields.

Shared error contract and handler

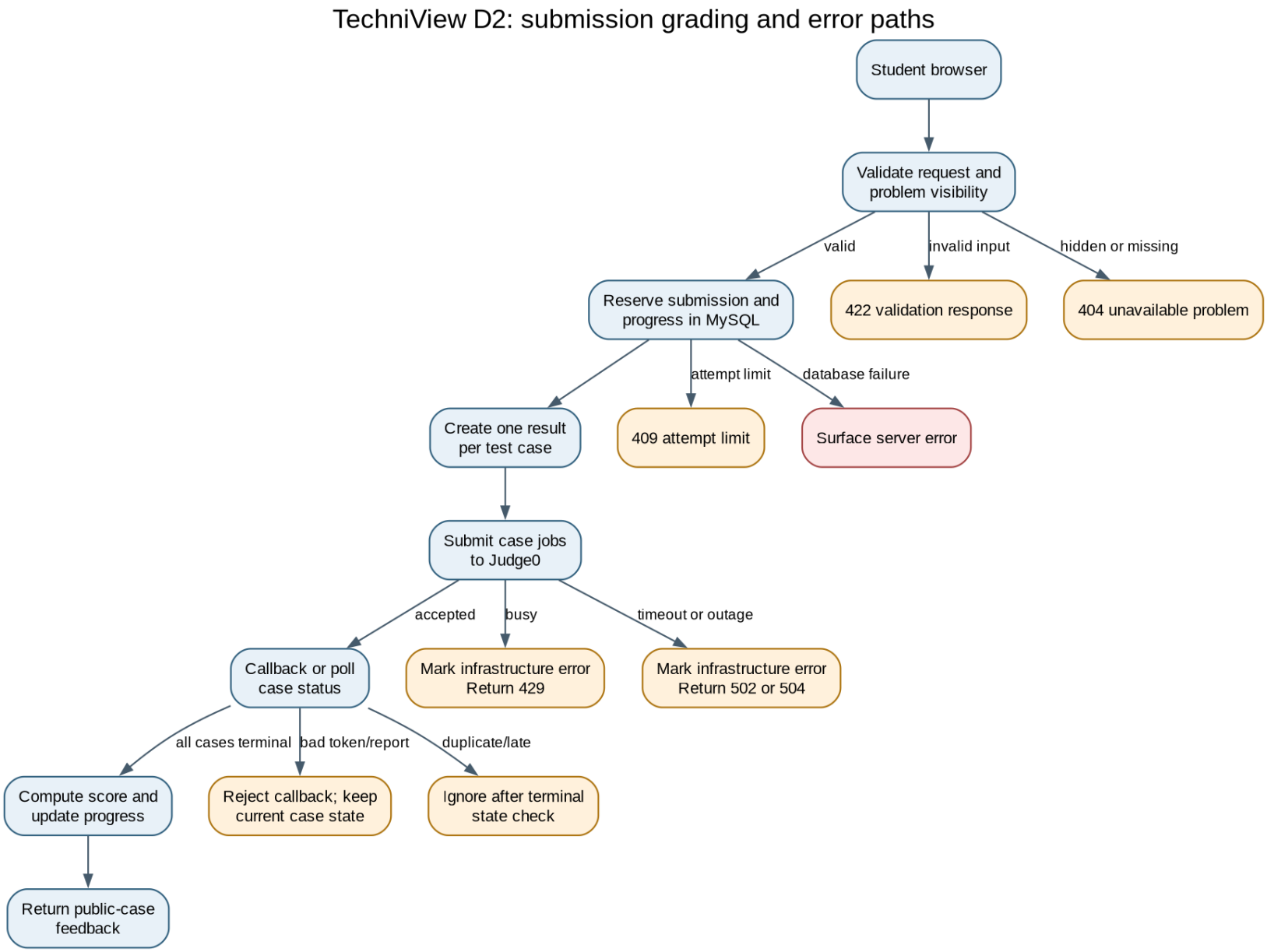
Request schema errors and application ApiError responses use {error: {code, message, details}}. Validation errors return 422 with field diagnostics. Authentication failures return 401. Role or membership failures return 403 or conceal a resource as 404. Missing resources return 404; invalid lifecycle transitions and attempt limits return 409. Runner timeouts return 504, busy responses return 429, and other runner failures return 502. Runner failures mark the affected case as infrastructure_error; they do not count as a valid student attempt. The shared handler is install_error_handlers in backend/app/core/errors.py. Direct HTTPException paths, including internal callback errors and runner failures, use FastAPI's detail response instead.

Grading flow, including failure paths

Student browser -> POST /api/submissions -> validate code and visibility -> reserve Submission and progress row in MySQL -> create one SubmissionCaseResult per case -> submit each case to Judge0 -> callback or owner polling updates each case -> finalize score and progress -> return public-case messages.

The diagram shows the failure outcomes as well as the successful path:

Figure 1. TechniView D2 grading and error paths. Project title: TechniView Design Diagram. Goal: create a web-based learning platform for technical computer science questions. Conventions: boxes are steps or services, arrows show request/data flow, and labeled arrows show failures and their handlers. These conventions apply to every D2 flow diagram.



```
flowchart TD
    A[Student browser] --> B[Validate request and problem visibility]
    B -->|invalid input| V[422 validation response]
    B -->|hidden or missing problem| N[404 response]
    B -->|valid| C[Reserve submission and progress in MySQL]
    C -->|attempt limit| L[409 response]
    C -->|database failure| DB[Surface server error]
    C --> D[Create one result per test case]
    D --> E[Submit case jobs to Judge0]
    E -->|busy| R[Mark infrastructure error and return 429]
    E -->|timeout or unavailable| X[Mark infrastructure error and return 504 or 502]
    E -->|accepted| F[Wait for callback or poll status]
    F -->|bad callback token/report| Q[Reject callback; retain current case state]
    F -->|duplicate or late callback| I[Ignore after terminal-state check]
    F -->|all cases terminal| G[Compute score and update progress]
    G --> H[Return result with public case feedback]
```

Failure branches:

- Invalid body or source length -> validation handler -> 422; student corrects the request.
- Hidden or unavailable problem -> visibility handler -> 404; no submission is created.
- No cases or attempt limit reached -> submission handler -> 409; professor must repair the problem or student must stop submitting.
- Judge0 busy -> mark case infrastructure error -> return 429; student may retry.
- Judge0 timeout or unavailable -> mark case infrastructure error -> return 504 or 502; student may retry after service recovery.
- Bad callback token or malformed report -> callback handler rejects report; valid callbacks can still complete the case.
- Duplicate or late callback -> row lock and terminal-state check keep the finalized grade unchanged.
- MySQL failure -> transaction fails and the API surfaces a server error; no automatic database retry is implemented.

Edge cases and error handling

The table gives one decision for each class category for each component. “Current” means the repository already implements the behavior. “Design” means the behavior is specified here and remains work if the code does not yet implement it. Recoverable means the user or operator can retry or correct the condition without restoring lost data.

ID	Component	Category	Edge case	Design decision
EC-01	Browser clients	Boundary values	Empty search, last page, or maximum page size	Current: API defaults to bounded page sizes and returns an empty list; UI displays the returned list. Handler: list route and page component. Recoverable: yes.
EC-02	Browser clients	Timing and concurrency	Session expires while a page request is in flight	Current: API returns 401; client surfaces generic request failure. Design: redirect to login and preserve the destination. Handler: AuthProvider. Recoverable: yes.
EC-03	Browser clients	Bad or missing input	Malformed form or API error	Current: browser required fields and server validation; login shows generic credentials error. Handler: form and FastAPI validation handler. Recoverable: yes.
EC-04	Browser clients	External dependencies	API is offline or slow	Current: fetch rejection/error is surfaced; no retry policy. Design: retain entered form data and offer explicit retry. Handler: page-level API error state. Recoverable: yes.

ID	Component	Category	Edge case	Design decision
EC-05	Authentication	Boundary values	Missing, expired, revoked, or disabled-user session	Current: 401 authentication_required or invalid_session. Handler: get_current_user. Recoverable: yes, sign in again.
EC-06	Authentication	Timing and concurrency	Logout races with an authenticated request	Current: request succeeds if it validated before revocation; later requests fail. Handler: session lookup and logout transaction. Recoverable: yes, sign in again.
EC-07	Authentication	Bad or missing input	Wrong credentials, mismatched signup passwords, duplicate email	Current: generic 401 for login, 422 for mismatch, 409 for existing email. Handler: auth and join routes. Recoverable: yes, correct input or use login.
EC-08	Authentication	External dependencies	MySQL unavailable during login/session validation	Current: database exception reaches framework server-error path. Design: return 503 and log correlation data without credentials. Handler: API exception middleware, not yet implemented. Recoverable: yes after DB recovery.
EC-09	Course and membership	Boundary values	Unknown join token, archived course, or last active instructor	Current: 404, 409 archived, 409 last_instructor. Handler: join and course routes. Recoverable: token/course issue requires instructor action; last instructor requires another instructor.
EC-10	Course and membership	Timing and concurrency	Two instructors concurrently remove/demote the last instructor	Current: count-then-write check is not serialized. Design: lock course membership changes in one transaction. Handler: course role/removal service; not yet implemented. Recoverable: yes, but fix is required before concurrent administration is trusted.
EC-11	Course and membership	Bad or missing input	User requests a course or membership they do not belong to	Current: 404 for missing membership/resource and 403 for insufficient staff role. Handler: authorization dependencies. Recoverable: yes if membership is granted.
EC-12	Course and membership	External dependencies	MySQL fails while joining or withdrawing	Current: transaction fails; no retry or user-safe database error mapping. Handler: route transaction and framework. Recoverable: yes after service recovery.
EC-13	Problem catalog and curriculum	Boundary values	Empty catalog result, unknown difficulty/tag, problem with no tests	Current: filters are validated; unavailable problems return 404; submission on no tests returns 409. Handler: problem and submission routes. Recoverable: catalog owner repairs missing content.
EC-14	Problem catalog and curriculum	Timing and concurrency	Problem archived after a page is loaded	Current: each read rechecks visibility; stale detail/submission fails with 404. Handler: _visible_problem. Recoverable: yes, reload and select another problem.
EC-15	Problem catalog and curriculum	Bad or missing input	Invalid id or filter values	Current: Pydantic/FastAPI validation returns 422. Handler: request validation handler. Recoverable: yes.

ID	Component	Category	Edge case	Design decision
EC-16	Problem catalog and curriculum	External dependencies	MySQL read fails	Current: server error; no fallback catalog is used at runtime. Handler: database session/framework. Recoverable: yes when MySQL returns.
EC-17	Authoring and question sets	Boundary values	Empty test set, duplicate problem IDs, invalid canonical solution	Current: publish validation rejects empty or failing content; set replacement rejects duplicate IDs. Handler: authoring route. Recoverable: yes, edit the draft.
EC-18	Authoring and question sets	Timing and concurrency	Two editors replace/publish the same draft concurrently	Current: last committed write wins; no version column or conflict check. Design: use optimistic versioning before multi-editor use. Handler: authoring service; not yet implemented. Recoverable: yes if detected before publication.
EC-19	Authoring and question sets	Bad or missing input	Unknown tag, inaccessible problem, extra payload field	Current: 422 for tag/payload errors and 404 for unavailable problems. Handler: authoring validation. Recoverable: yes.
EC-20	Authoring and question sets	External dependencies	Judge0 is down during canonical-solution validation	Current: validation catches runner exceptions and returns an issue; draft remains unpublished. Handler: validate_problem. Recoverable: yes, retry validation.
EC-21	Assignment management	Boundary values	Empty question set, due date before available date, no active recipients	Current: empty set and invalid dates are rejected; zero recipients is allowed. Handler: assignment creation/publish. Recoverable: yes, correct set/date or enroll students.
EC-22	Assignment management	Timing and concurrency	Membership changes while recipients are snapshotted	Current: creation captures active student memberships in its transaction; later joiners are not added automatically. Handler: assignment creation. Recoverable: manual assignment recreation; product rule is an open question.
EC-23	Assignment management	Bad or missing input	Question set is absent, not owned, or not published	Current: concealed 404. Handler: create_assignment_from_question_set. Recoverable: yes, choose an owned published set.
EC-24	Assignment management	External dependencies	MySQL fails between assignment and recipient inserts	Current: transaction rollback prevents partial persistence. Handler: SQLAlchemy transaction. Recoverable: yes, retry after recovery.
EC-25	Submission grading and progress	Boundary values	Source empty/too large, no cases, or attempt limit reached	Current: 422 for source validation and 409 for no tests/limit. Handler: SubmissionCreate and create_submission. Recoverable: correct input or stop.
EC-26	Submission grading and progress	Timing and concurrency	Duplicate callbacks or simultaneous callbacks finalize one submission	Current: row locks and terminal-state check make finalization idempotent. Handler: callback and _finalize_submission. Recoverable: yes, repeated delivery is safe.

ID	Component	Category	Edge case	Design decision
EC-27	Submission grading and progress	Bad or missing input	Callback token unknown, mismatched, or report malformed	Current: callback rejects with 400/404/409; it does not change the grade. Handler: receive_judge0_callback. Recoverable: runner can deliver a valid callback or polling can retrieve status if token exists.
EC-28	Submission grading and progress	External dependencies	Judge0 busy, times out, or returns invalid data	Current: case becomes infrastructure_error; API returns 429/504/502. Handler: _mark_case_infrastructure_error and _judge0_error. Recoverable: yes; infrastructure failures do not consume attempts.
EC-29	Analytics	Boundary values	No assigned rows, offset beyond total, or limit at bounds	Current: zero counts and empty page; limit is 1..100. Handler: analytics routes and calculate_metrics. Recoverable: yes.
EC-30	Analytics	Timing and concurrency	Progress changes while aggregate query runs	Current: database transaction provides its configured isolation view; subsequent read reflects later state. Handler: MySQL transaction. Recoverable: yes, refresh.
EC-31	Analytics	Bad or missing input	Student requests staff report or problem not assigned to course	Current: 403 or 404. Handler: StaffMembership and analytics route. Recoverable: yes if access/assignment is corrected.
EC-32	Analytics	External dependencies	MySQL unavailable or aggregate query is too slow	Current: server error or timeout; no cache or rollup fallback is wired. Handler: DB driver/API; optimization is open. Recoverable: retry after recovery.
EC-33	API runtime and configuration	Boundary values	Invalid port, cookie days, or empty CORS origins	Current: integer parse fails at startup for invalid numbers; empty origin tuple blocks browser origins. Handler: Settings initialization. Recoverable: operator corrects environment and restarts.
EC-34	API runtime and configuration	Timing and concurrency	API process restarts during an in-flight request	Current: client receives connection failure; database transaction either commits or rolls back. Handler: client retry and SQL transaction. Recoverable: usually; submission outcome must be checked before resubmission.
EC-35	API runtime and configuration	Bad or missing input	Unsupported content type or invalid JSON	Current: FastAPI returns validation/client error. Handler: framework and installed validation handler. Recoverable: yes.
EC-36	API runtime and configuration	External dependencies	MySQL, Redis, or other configured dependency is absent	Current: route-specific failure; startup does not wait for every dependency outside Compose. Handler: dependency call or health route. Recoverable: yes after dependency returns.
EC-37	MySQL persistence	Boundary values	Unique constraint or FK violation	Current: SQLAlchemy/database rejects write; most integrity errors are not mapped to a stable API code. Handler: MySQL constraint and framework error path. Recoverable: yes after correcting conflicting data.

ID	Component	Category	Edge case	Design decision
EC-38	MySQL persistence	Timing and concurrency	Deadlock or lock wait timeout	Current: no application retry policy. Design: retry only idempotent transaction once, otherwise surface 503. Handler: transaction boundary; retry is not implemented. Recoverable: yes.
EC-39	MySQL persistence	Bad or missing input	Migration/schema mismatch or invalid enum	Current: migration/startup or write fails; no compatibility fallback. Handler: Alembic/deployment operator. Recoverable: yes with corrected migration/deployment.
EC-40	MySQL persistence	External dependencies	Database outage or disk exhaustion	Current: no local persistence fallback. Health endpoint reports failure; request fails. Handler: DB driver and operator. Recoverable: after service/backup recovery.
EC-41	Judge0 runner	Boundary values	CPU/memory limit reached, empty output, or source execution exception	Current: Judge0 terminal status is translated to a submission result. Handler: wrapper and _error_for_status. Recoverable: student can change code; resource cap requires problem author change.
EC-42	Judge0 runner	Timing and concurrency	Worker queue delay or callback arrives after polling/finalization	Current: queued/running status remains visible; late callback is idempotently ignored after terminal state. Handler: callback/polling path. Recoverable: yes unless service stalls.
EC-43	Judge0 runner	Bad or missing input	Invalid execution wrapper or malformed runner response	Current: malformed response becomes infrastructure error and 502. Handler: submission route. Recoverable: yes after correcting content or runner.
EC-44	Judge0 runner	External dependencies	Judge0, its PostgreSQL, or Redis is unavailable	Current: API surfaces 429/502/504; health routes report component status. Handler: judge0 client and health router. Recoverable: yes after service recovery.
EC-45	Health and operations	Boundary values	Health check called with shallow/deep setting	Current: shallow Judge0 probe uses /about; deep probe submits print(42). Handler: health router. Recoverable: yes; deep test may consume runner capacity.
EC-46	Health and operations	Timing and concurrency	Probe overlaps deploy or service restart	Current: reports transient failure; no alert debounce or readiness gate is defined. Handler: health route only. Recoverable: yes; operator should recheck.
EC-47	Health and operations	Bad or missing input	Unknown health route or invalid deep value	Current: 404 or FastAPI boolean validation. Handler: FastAPI. Recoverable: yes.
EC-48	Health and operations	External dependencies	Probe itself cannot reach a service	Current: response includes ok=false and limited error text; no alert is sent. Handler: check function. Recoverable: yes after service recovery; detection is manual.

ID	Component	Category	Edge case	Design decision
EC-49	Notifications	Boundary values	No recipient, notification preferences, or event type	Current: no notification is generated. Design: do not enqueue an empty message; surface state in the API/UI. Handler: assignment/submission state read. Recoverable: no notification to recover; feature gap.
EC-50	Notifications	Timing and concurrency	Assignment changes while a notification would be sent	Current: no notification worker or delivery race. Design: if added, send from a transactional outbox after commit. Handler: proposed notification service. Recoverable: design work required before implementation.
EC-51	Notifications	Bad or missing input	Missing address or invalid message payload	Current: no delivery interface exists. Design: reject the event at producer and record a typed error; no current handler. Recoverable: design work required.
EC-52	Notifications	External dependencies	Mail or push provider unavailable	Current: no provider configured. Design: bounded retry with dead-letter visibility after provider is selected. Handler: proposed worker. Recoverable: design work required; no retry exists today.
EC-53	Logging	Boundary values	Log event exceeds the configured runtime output limit	Current: no application event schema or size policy exists. Design: cap field lengths and omit source code, passwords, tokens, and callback secrets. Handler: proposed logging formatter. Recoverable: yes; event may be truncated, never the secret.
EC-54	Logging	Timing and concurrency	Multiple workers emit events concurrently	Current: container runtime collects process output; ordering across workers is not guaranteed. Design: include UTC timestamp, request/submission id, and process identity. Handler: proposed structured formatter. Recoverable: yes; order may be ambiguous.
EC-55	Logging	Bad or missing input	Untrusted user text contains control characters or secret values	Current: no application logging policy is implemented. Design: sanitize control characters and use an allowlist of fields. Handler: proposed logging formatter. Recoverable: yes; rejected fields are omitted.
EC-56	Logging	External dependencies	Container log collector is unavailable or storage fills	Current: no application-owned log sink or fallback exists. Design: keep request handling independent of log delivery and report collector failure through deployment monitoring. Handler: container runtime/operator. Recoverable: partially; missing logs cannot be reconstructed.

Requirements traceability table

US and AC rows come from User_Stories.md. NFR rows are candidate requirements because the course has not supplied numeric performance, availability, accessibility, retention, or compliance targets.

ID and short text	Priority	Design elements	Edge cases and figures	How and gaps
US-01: Students access and run coding problems in a web environment	Must	Problem, ProblemTestCase; GET /api/problems; POST /api/submissions; build_case_execution; browser problem routes	EC-13 to EC-16, EC-25 to EC-28, EC-41 to EC-44; D1 Figure 1; D2 Figure 1	API serves statements and submits Python source to Judge0. Browser has problem/detail routes. Gap: frontend flow and language support are limited to Python; no local setup is needed.
AC-01.1: A selected problem submission runs against cases and returns feedback	Must	Submission, SubmissionCaseResult; POST/GET /api/submissions; Judge0 callback; submission response	EC-25 to EC-28, EC-41 to EC-44; D1 Figure 1; D2 Figure 1	Per-case results produce aggregate score and public-case messages. Hidden cases affect score, but expected outputs are not returned.
US-02: Students track progress and performance	Must	StudentPracticeProgress, StudentAssignmentProgress; GET /api/me/analytics; student statistics route	EC-26, EC-29 to EC-32; D1 Figure 1; D2 Figure 1	Finalization updates progress and analytics returns counts and error metrics. Gap: no retention window or performance target is specified.
US-03: Professors assign/manage problems and assess students	Must	CourseMembership, QuestionSet/QuestionSetItem, Assignment/AssignmentItem/AssignmentRecipient; assignment, staff submission, and analytics routes	EC-09 to EC-12, EC-21 to EC-24, EC-29 to EC-32; D1 Figure 1	Professors author, publish, inspect submissions, and read analytics. Gaps: notifications absent; membership/recipient behavior needs a product decision.
US-04: Problem team creates and curates quality problems	Should	Problem, ProblemTestCase, ProblemTag; authoring validate/publish routes; catalog importer	EC-13 to EC-20; no authoring figure	Draft validation runs canonical solutions against all cases. Gap: professors perform authoring; no separate problem-management role exists.
US-05: Protect student data and submissions	Must	UserSession.token_hash; scrypt password_hash; membership authorization; hidden test visibility; Judge0 isolation	EC-05 to EC-08, EC-11, EC-34, EC-39, EC-53 to EC-56; D1 Figure 1; D2 Figure 1	Tokens are hashed and cookies are HTTP-only; hidden cases and canonical solutions are withheld. Gaps: legal scope, retention, transport, backups, recovery, and security review.
NFR-01: Isolate untrusted submitted code	Must, candidate	Judge0 worker container; CPU/memory limits; internal callback token hash	EC-41 to EC-44; D2 Figure 1	Runner is separate and privileged. Deployment hardening and threat review remain open.
NFR-02: Preserve submission/progress consistency under duplicate callbacks	Must, candidate	SubmissionCaseResult composite key; row locks; terminal-state guard; finalize_progress	EC-26 to EC-28, EC-38; D1 Figure 1; D2 Figure 1	Duplicate/late callbacks do not change finalized results or double-count progress. Lock/deadlock retry is not implemented.
NFR-03: Keep user-facing errors actionable and stable	Should, candidate	ApiError; install_error_handlers; 401/403/404/409/422 mappings	EC-03, EC-07, EC-08, EC-15, EC-19, EC-27, EC-35, EC-37; D2 Figure 1	API errors have code/message/details. Direct HTTPException and infrastructure errors differ; unhandled DB errors are not normalized.
NFR-04: Operators can detect dependency failures	Should, candidate	GET /api/health and dependency probes	EC-36, EC-44 to EC-48; no health figure	Probes cover MySQL and Judge0 dependencies. There is no alerting, dashboard, or health history.
NFR-05: Keep analytics usable as course size grows	Should, candidate	Analytics queries, limit/offset, progress aggregates	EC-29 to EC-32; no analytics figure	Results are grouped in Python before pagination. No workload or latency target exists; query scalability needs measurement.
NFR-06: Meet accessibility and regulatory requirements	Must, candidate from hidden stakeholder	React pages; User disabled state; session controls	EC-05 to EC-08, EC-53 to EC-56; no accessibility/compliance figure	No accessibility audit, jurisdiction, retention, consent model, or compliance mapping exists. Cannot claim this is satisfied until scope and criteria are agreed.

Peer review did not change a design decision. DR-01 and DR-10 record repository-audit documentation corrections; peer suggestions are follow-up actions.

Reverse traceability check. Every implemented domain component maps to US-01 through US-05, AC-01.1, or a candidate NFR. Health checks map to NFR-04; Judge0 isolation maps to US-01 and NFR-01; sessions and authorization map to US-05/NFR-06; authoring maps to US-04; assignment and analytics tables map to US-02/US-03. Notifications have no requirement or implementation and remain explicitly out of current scope. Demo seed data, import scripts, and health diagnostics are supporting operations, not separate product scope. No current API component is left without a requirement link.

Design rationale

ID	Decision and trade-off
DR-01	We kept the D0 split between User Data and Question data as logical table groups in one MySQL database because D1 and the running Compose stack use one platform database. We accepted weaker physical isolation because the current project does not operate two application databases. This clarifies the D0 cylinders after review finding R-01.
DR-02	We chose relational tables and foreign keys for memberships, assignment recipients, ordered content, and submissions over embedded JSON because the implementation relies on joins, unique constraints, and transactions. We accepted migration work because invalid cross-course references would corrupt grades and rosters. This extends the D0 database decision log.
DR-03	We chose to store only a hash of random session and join tokens over storing reusable secrets because a database read should not reveal a valid token. We accepted the need to reissue a lost token because recovery by lookup would weaken that protection.
DR-04	We chose a separate Judge0 service over executing student code in FastAPI because hostile code needs CPU and memory isolation. We accepted privileged containers and a separate PostgreSQL/Redis stack because the project does not build its own sandbox. This references the D0 Code Runner separation decision.
DR-05	We chose per-case jobs with callback tokens and polling over one synchronous whole-submission call because cases can finish independently and clients can close while work continues. We accepted more callback state and failure handling because durable per-case outcomes support retries and audit.
DR-06	We chose immutable published problems and question sets over editing in place because assignments need stable content after students begin. We accepted copy-and-republish work because changing historical questions could alter grading context.
DR-07	We chose assignment-recipient snapshots over recomputing recipients on every read because the assignment's original audience should be explicit. We accepted that later joiners are not automatically included because the current product has no add-recipient workflow.
DR-08	We chose cookie sessions with server-side revocation over browser-stored bearer tokens because the browser can send an HTTP-only cookie without exposing it to normal page scripts. We accepted CSRF and production-cookie configuration work because the current deployment is a local development stack.
DR-09	We chose no notification subsystem over adding email or push without event, preference, and privacy rules. We accepted that students must check the application because no delivery requirements or provider were specified.
DR-10	We chose to retain D1's historical scope and add a current implementation map here over rewriting its original design snapshot. We accepted two documents with different dates because the design record should show what D1 specified and what the code later added. Review finding R-02 prompted a note and duplicate-index cleanup in D1.

Review corrections and change log

Review ID	Finding	Documentation change
R-01	D0's separate database cylinders could be read as two deployed TechniView databases. The current Compose file has one platform MySQL database; Judge0 has separate PostgreSQL and Redis.	Added an implementation note to D0 explaining that its application cylinders are logical data groups. DR-01 records why.
R-02	D1 is a commit-based snapshot and its index table repeated identical index entries. Without a status note, readers could mistake it for the current full schema.	Added a current-scope note and per-diagram title/conventions to D1 and removed duplicate index rows. D1 remains a historical design snapshot; D2 lists the current schema. DR-10 records why.
R-03	D0 and D1 do not describe the current routes for authoring, joins, assignments, staff review, curriculum, or health.	Added current route inventory and models to D2. No older diagram was redrawn beyond the D0 logical-database clarification.

Open design questions

ID	Question	Owner to resolve
OQ-01	Are assignment recipients a permanent snapshot, or should later students and reactivated members be added?	Derek Corniello
OQ-02	Should withdrawn students remain in assignment analytics and historical recipient lists?	Jason Bellerjeau
OQ-03	What notification events, channels, preferences, and delivery guarantees are required?	Ryan Sippy
OQ-04	Which privacy rules, legal jurisdiction, retention periods, and account deletion behavior apply?	Team
OQ-05	What are the production TLS, secure-cookie, CORS, secret rotation, backup, and restore targets?	Derek Corniello
OQ-06	What latency and volume targets should govern analytics and Judge0 capacity?	Team
OQ-07	Should the API normalize FastAPI HTTPException and database failures into the ApiError contract?	Derek Corniello
OQ-08	Is Python-only execution the intended product requirement or an implementation limitation?	Team

Week 8 to Week 9 plan

The design is ready for a review discussion. Walk through the architecture and capture any useful suggestions for the implementation handoff.

When	Work	Owner	Completion evidence
Week 8 review	Walk through the architecture, error paths, and traceability table; record useful follow-up suggestions.	Team	Peer_Review_Notes.md captures the discussion and agreed actions.
Week 8	Decide late-enrollment and analytics behavior, and confirm remaining design details.	Jason Bellerjeau and Derek Corniello	Written decisions and updated D2 rationale or tables.
Week 8	Confirm security, privacy, and notification scope with the instructor/team.	Team	Approved wording or explicitly deferred items.
Week 9	Turn accepted D2 rows into implementation tasks and a technical specification traceability matrix.	Team	Each approved requirement has an implementation task and verification method.
Week 9	Measure analytics and runner response under the agreed workload before setting capacity claims.	Derek Corniello	Recorded workload, response times, queue depth, and chosen limits.

Peer design review notes

Overall, this was easy to follow. The D0-to-D2 progression, grading error-path diagram, and traceability table give the team a clear starting point for implementation. I did not find a reason to hold up the design. The follow-ups are small planning items.

Peer design review notes

I liked how the D0, D1, and D2 views fit together. The comments below are positive, low-priority follow-ups from the peer review.

ID	Criterion	Feedback and reviewer	Action	Owner	Priority	Due date
PR-01	Clarity	The D0-to-D2 progression is clear. The note about the two D0 database cylinders being logical groups helped me understand how the diagram matches the current setup. Raised in peer review.	Keep that note beside the D0 conventions when the diagram is reused.	Derek Corniello	Worth considering	Week 9 technical specification
PR-02	Completeness	D2 fills in route and model details that were not in the older D1 snapshot. The snapshot label made the difference easy to understand. Raised in peer review.	Carry the route list into the technical specification and update it if routes change.	Jason Bellerjeau	Worth considering	Week 9 technical specification
PR-03	Feasibility	The grading failure path is helpful. It made the runner boundary and fallback behavior easier to talk about. Raised in peer review.	Check timeout and retry limits against the deployed Judge0 configuration.	Derek Corniello	Worth considering	Week 9 technical specification

Feedback and review plan

ID	Criterion	Feedback and reviewer	Action	Owner	Priority	Due date
PR-04	Risk	The edge-case table gives the team a good place to start. Membership changes and production security settings are worth keeping on the checklist. Raised in peer review.	Keep these items visible in the implementation plan.	Derek Corniello	Worth considering	Before production deployment
PR-05	Completeness	I liked that the requirements table points to specific design details and checks scope in reverse too. Raised in peer review.	Use it as the starting traceability matrix for Week 9.	Ryan Sippy	Worth considering	Week 9 technical specification
PR-06	Clarity	The open questions are easy to spot, which should make the review discussion more focused. Raised in peer review.	Talk through late enrollment and notification behavior, then record the decisions.	Team	Worth considering	Week 8 review

Summary of feedback

Clarity came up most in my notes. I also liked the error-path diagram, edge-case coverage, and links from requirements to design details. I did not suggest any design changes.

Two questions I raised in the review

1. If the runner accepts a job but never returns a result, what timeout and retry limits should the implementation use?
2. If a student joins or rejoins after an assignment is published, should they receive it, and how should that show up in analytics?

Plan for the review slot

Spend about one third of the slot walking through the architecture, grading failure path, and traceability table. Use the rest for questions about edge cases, runner behavior, and late enrollment. Write down useful suggestions with a criterion, action, owner, priority, and due date. Keep the conversation relaxed.